

Programming Assignment 3

Posted: 2/16/2025 Due Date: 2/23/2025

There are the following objectives in this assignment.

1. Get familiar with pointer and reference
2. Basic flow control (loop, etc.)
3. Implement basic algorithms

There are three components of this assignment. Each component asks you to write one C++ source file with the following names. I recommend you start with the provided skeleton code.

1 ECRemoveMatch.cpp

The purpose of this assignment is to practice pointers and references. Here, you are given an array of pointers to strings. You need to implement a function called **removeMatch** that takes this array of pointers to strings and removes any matches with a duplicate pointer. For example, suppose you are given an array of 3 pointers:

```
string s1 = "A", s2 = "A"  
string *p1 = &s1, p2 = &s2, p3 = &s1
```

The array of pointers: `array[] = {p1, p2, p3};`

If you invoke: `removeMatch(array, &s1)`, then the array should look like: `array[] = {p2}` since the strings pointed to are s1, s2, and s1 respectively.

Note: I will use C++ STL vector to represent an array. Essentially, STL vector is a “better” array. I want you to start to experiment with this powerful utility class. Google it to find references on how to use STL vector.

2 ECSplitIntervals.cpp

You are given a vector of intervals (pairs of exactly 2 integers i.e. [1, 3] or [5,5]) sorted by their beginning (first) value. An interval [a, b] is the set of all integers from a to b (inclusive). You are to write a function called **splitIntervals** that splits all intervals that include any integers in a given vector.

For example, suppose $A = [[1,5]]$ and $B = [3]$.
Then, after `splitIntervals` is run, $A = [[1,2],[4,5]]$.

As another example, suppose $C = [[1,8],[7,12]]$ and $D = [3,5]$.

Then, after `splitIntervals` is run, $C = [[1,2],[4,4],[6,8],[7,12]]$.

Now, neither 3 nor 5 are present in any of C 's intervals.

Note that the interval $[1,8]$ was split multiple times, while $[7,12]$ was not split.

For the purposes of this assignment, your vector does not need to be in any particular order after `splitIntervals` is called.

3 **ECNumOccurrences.cpp**

You are given integer vectors A and B . You want to find the amount of times a number in A occurs in B . For this, you are to implement a function called **NumOccurrences**, which takes two vectors and returns a vector C (whose indices map to A and contains the number of occurrences). Note that B is sorted before being passed into the function.

For example, suppose $A = \{1, 3\}$ and $B = \{1, 1, 2, 3, 6, 7, 10\}$.

Then, after `NumOccurrences` is run, $C = \{2, 1\}$ indicating that “1” from A occurs twice and “3” occurs once.

As another example, suppose $A = \{1, 3, 1\}$ and $B = \{1, 1, 1, 3, 3, 6, 7\}$.

Then, after `NumOccurrences` is run, $C = \{3, 2, 3\}$ indicating that “1” from A occurs thrice, “3” occurs twice, and “1” occurs thrice again.

Things to consider when implementing this function:

1. You must implement the function as efficiently as possible. Recall that efficiency is one of the most important aspects of C++ programming. You should carefully think about the algorithm you are to use. The most straightforward algorithm is simply comparing each pair of numbers in the two vectors. However, this is very slow when the vectors get large. We will have run-time checks to ensure your code meets this criteria.
2. Don't reinvent the wheel! If you can, try to use the provided standard library functions. Search them up if you are not familiar with them.

Instructions for Submission

Please read the following carefully.

1. Submit your programs in GradeScope. A link is provided from HuskyCT. Note that you can submit multiple files by drag and drop into the submission UI.
2. You must use the file names as specified. If you change the filename, the Autograder won't work.
3. Please don't create your own files. Autograder won't know about the files you created (not in the provided starter code) and won't run test cases properly.
4. For automated grading, we mainly focus on functionalities. However, sometimes we will also test on (i) efficiency, (ii) code complexity, and (iii) code quality. We may use third-party tools for some of these tasks. While these tools have been around for some time, they can still sometimes produce not so accurate report about your code. If this is the case, contact the TAs about the issues.
5. Submit **ECRemoveMatch.cpp**, **ECSplitIntervals.cpp**, and **ECNumOccurrences.cpp**